

LAROURA

BTS 2 — Option SISR | Mohamed

DOSSIER TECHNIQUE

MISE EN PLACE D'UNE ARCHITECTURE RÉSEAU

DMZ avec Docker sur VM Proxmox — Firewall, Nginx & Clients

Candidat	Mohamed LAROURA
Formation	BTS SIO — Option SISR (2ème année)
Évaluation	Docker — Réseaux — BTSSIO-02 — 25-26
Environnement	VM Ubuntu 22.04.5 LTS sur Proxmox VE 9.1.1
Technologie	Docker, Debian (iptables), Nginx, Alpine Linux

Fiche de réalisation professionnelle

Projet B — Mise en place d'une architecture DMZ

Identification de la réalisation

Intitulé	Mise en place d'une architecture réseau sécurisée avec Docker (DMZ) sur VM Proxmox
Candidat	LAROURA Mohamed — N° 02248898094
Centre de formation	CFA ESTIAM — Paris
Option	BTS SIO SISR — Session 2026
Période de réalisation	01/09/2025 au 30/04/2026
Contexte	Réalisation en cours de formation
Type	Réalisation individuelle
Contribution personnelle	100 % — Conception, déploiement, validation et documentation

Productions et documents associés

- Dossier technique (le présent document PDF)
- Schéma d'architecture WAN / DMZ / LAN (document parallèle)
- Plan d'adressage réseau (fichier Excel)
- Captures d'écran des tests (curl, iptables -L, docker ps, ping)
- Commandes de création des réseaux et conteneurs Docker
- Règles iptables documentées (ACCEPT HTTP, DROP WAN → LAN)

Compétences E5 mobilisées (référentiel BTS SIO 2026)

■	Gérer le patrimoine informatique
■	Répondre aux incidents et aux demandes d'assistance et d'évolution
■	Développer la présence en ligne de l'organisation
■	Travailler en mode projet
■	Mettre à disposition des utilisateurs un service informatique
■	Organiser son développement professionnel

Table des matières

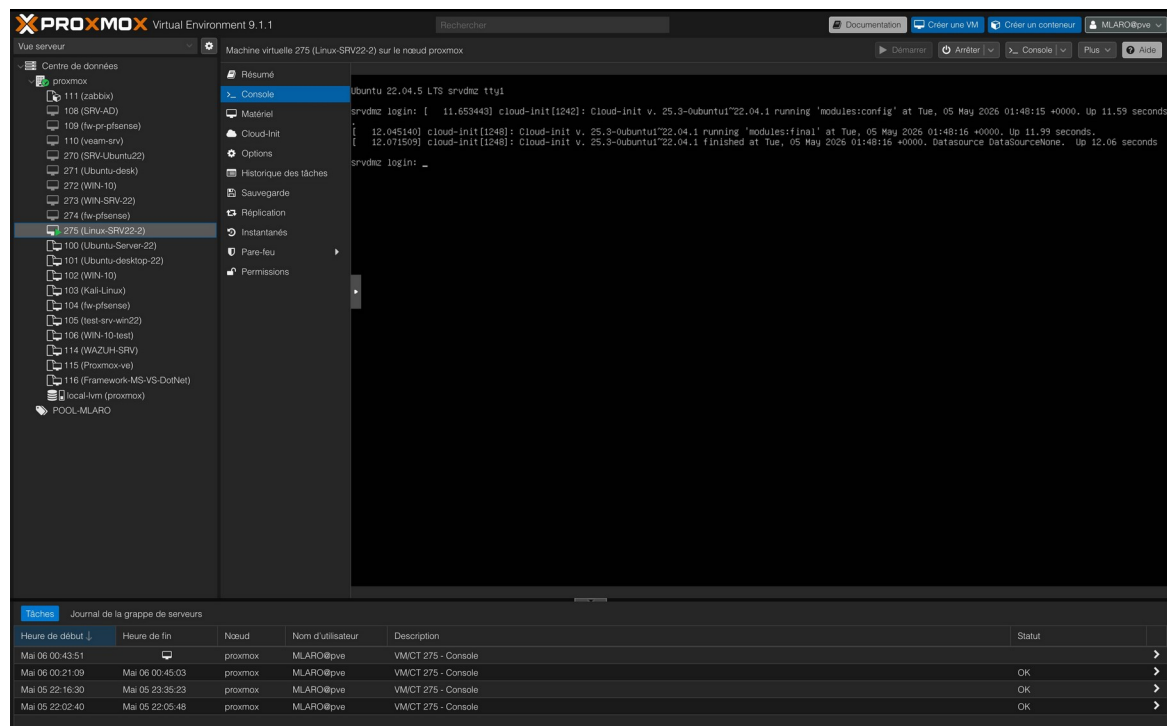
Fiche de réalisation professionnelle	2
Étape 0 : Environnement de travail	4
Console Proxmox	4
Connexion SSH	4
Vérification de Docker	4
Étape 1 : Création des réseaux	6
Architecture réseau	6
Commandes	6
Schéma d'architecture WAN / DMZ / LAN	7
Étape 2 : Configuration du Pare-feu	8
Lancement du conteneur Firewall	8
Installation d'iptables	8
Règles iptables	9
Étape 3 : Mise en place du Serveur Web (DMZ)	10
Étape 4 : Déploiement des Clients	11
Étape 5 : Tests de validation	12
Test 1 : LAN vers DMZ (HTTP)	12
Test 2 : WAN vers DMZ (HTTP)	13
Test 3 : Isolation WAN vers LAN	14
Récapitulatif final	15
Analyse Critique et Compétences SISR	16
Annexe A — Plan de tests formalisé	17
Annexe B — Persistance des règles et rollback	18
Annexe C — Veille et référentiels normatifs	19
Annexe D — Cartographie détaillée des compétences E5	20
Conclusion	21

Étape 0 : Environnement de travail

Ce projet est réalisé sur une machine virtuelle Ubuntu 22.04.5 LTS (VM 275) hébergée sur l'hyperviseur Proxmox VE 9.1.1. L'accès se fait via SSH depuis un terminal macOS.

Console Proxmox

La VM est visible dans l'interface Proxmox sous le nom « srvdmz » :



Connexion SSH

Connexion à la VM depuis le terminal Mac via SSH :

```
ssh manager@172.16.1.13
```

```
Last login: Mon May 4 23:57:02 on tty000
mohamed.silMacBook-Air-de-Mohamed ~ % ssh manager@172.16.1.13
manager@172.16.1.13's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-177-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of mar. 05 mai 2026 20:24:58 UTC

System load: 0.0          Processes: 121
Usage of /: 15.7% of 48.91GB Users logged in: 0
Memory usage: 7%          IPv4 address for ens18: 172.16.1.13
Swap usage: 0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.
https://ubuntu.com/engage/secure-kubernetes-at-the-edge

La maintenance de sécurité étendue pour Applications n'est pas activée.

1 mise à jour peut être appliquée immédiatement.
1 de ces mises à jour est une mise à jour de sécurité.
Pour afficher ces mises à jour supplémentaires, exécuter : apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Tue May 5 01:49:37 2026 from 172.16.255.254
manager@srvdmz:~$
```

Vérification de Docker

```
docker --version
```

```
manoger@srvdhtz:~$ docker --version  
Docker version 29.1.3, build 29.1.3-0ubuntu3-22.04.1  
manoger@srvdhtz:~$
```

Docker version 29.1.3 est installé et fonctionnel sur la VM.

Étape 1 : Création des réseaux

Création de 3 réseaux virtuels Docker isolés représentant les zones WAN, DMZ et LAN.

Architecture réseau

Réseau	Sous-réseau	Zone	Rôle
wan_net	172.30.0.0/24	WAN	Accès externe
dmz_net	172.20.0.0/24	DMZ	Serveur Web
lan_net	172.10.0.0/24	LAN	Clients internes

Commandes

```
sudo docker network create --subnet 172.30.0.0/24 wan_net
sudo docker network create --subnet 172.20.0.0/24 dmz_net
sudo docker network create --subnet 172.10.0.0/24 lan_net
```

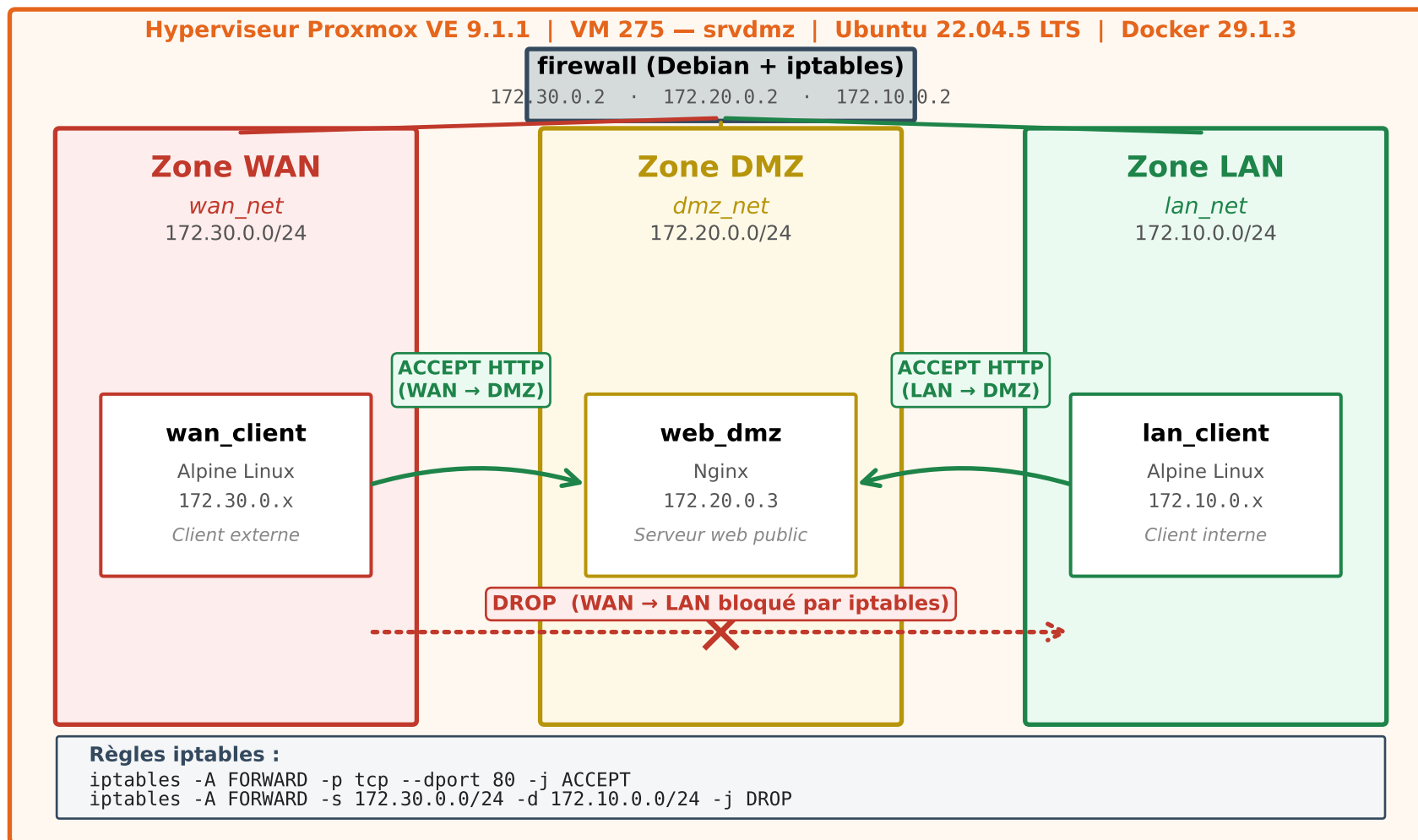
Vérification avec docker network ls :

```
manager@srvdmz:~$ sudo docker network create --subnet 172.30.0.0/24 wan_net
[sudo] password for manager:
6d54aaf1bb89d841bb8672c9df2237fad97c262b5405054ecd6d9e1ead9bc
manager@srvdmz:~$ sudo docker network create --subnet 172.20.0.0/24 dmz_net
eff96c1b520b3b9e15e6154abc380544e8c289a38dc3f189f5be9456c4b33f
manager@srvdmz:~$ sudo docker network create --subnet 172.10.0.0/24 lan_net
c9bbae6d6c37991ff14928740e556551fb27b5d91564a0405eda02fd2cb607
manager@srvdmz:~$ sudo docker network ls
NETWORK ID   NAME      DRIVER  SCOPE
24e58189d91  bridge   bridge  local
eff96c1b520e  dmz_net   bridge  local
24e58189d91a  host     host    local
c9bbae6d6d3   lan_net   bridge  local
58e7071e692a  none     null    local
6d54aaf1bb8b  wan_net   bridge  local
manager@srvdmz:~$
```

Les trois réseaux wan_net, dmz_net et lan_net sont créés avec succès.

Schéma d'architecture — Projet B

Architecture réseau segmentée WAN / DMZ / LAN avec firewall iptables



Règles iptables

Configuration des règles de filtrage : autoriser le HTTP (port 80) et bloquer le WAN vers le LAN.

```
sudo docker exec firewall iptables -A FORWARD -p tcp --dport 80 -j ACCEPT
sudo docker exec firewall iptables -A FORWARD -s 172.30.0.0/24 -d 172.10.0.0/24 -j DROP
```

Vérification des règles avec iptables -L -v :

```
manager@srvdhtz:~$ sudo docker exec firewall iptables -A FORWARD -p tcp --dport 80 -j ACCEPT
manager@srvdhtz:~$ sudo docker exec firewall iptables -A FORWARD -s 172.30.0.0/24 -d 172.10.0.0/24 -j DROP
manager@srvdhtz:~$ sudo docker exec firewall iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source            destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source            destination
  0      0 ACCEPT     tcp  --  any    any    anywhere         tcp dpt:http
  0      0 DROP      all  --  any    any    172.30.0.0/24    dsl:172-10-0-0.dsl.sndg02.sbcglobal.net/24
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source            destination
manager@srvdhtz:~$
```

Les deux règles sont bien actives : ACCEPT pour le trafic HTTP et DROP pour le WAN vers le LAN.

Étape 3 : Mise en place du Serveur Web (DMZ)

Lancement du serveur Nginx dans la zone DMZ et configuration des routes de retour pour qu'il puisse joindre le LAN et le WAN via le firewall.

Commandes

```
sudo docker run -dit --name web_dmz --cap-add NET_ADMIN --network dmz_net nginx
sudo docker exec web_dmz ip route add 172.10.0.0/24 via 172.20.0.2
sudo docker exec web_dmz ip route add 172.30.0.0/24 via 172.20.0.2
```

Vérification des routes avec ip route :

```
manager@srvdmz:~$ sudo docker exec web_dmz ip route add 172.10.0.0/24 via 172.20.0.2
manager@srvdmz:~$ sudo docker exec web_dmz ip route add 172.30.0.0/24 via 172.20.0.2
manager@srvdmz:~$ sudo docker exec web_dmz ip route
default via 172.20.0.1 dev eth0
172.10.0.0/24 via 172.20.0.2 dev eth0
172.20.0.0/24 dev eth0 proto kernel scope link src 172.20.0.3
172.30.0.0/24 via 172.20.0.2 dev eth0
manager@srvdmz:~$
```

Les routes de retour vers le LAN (172.10.0.0/24) et le WAN (172.30.0.0/24) passent bien par le firewall (172.20.0.2).

Étape 4 : Déploiement des Clients

Création des machines de test dans les zones LAN et WAN pour valider les accès à travers le firewall.

Récapitulatif des conteneurs

Conteneur	Image	Réseau	Rôle
firewall	debian	wan + dmz + lan	Routage & filtrage iptables
web_dmz	nginx	dmz_net	Serveur Web en DMZ
lan_client	alpine	lan_net	Client interne (test LAN)
wan_client	alpine	wan_net	Client externe (test WAN)

Commandes

```
sudo docker run -dit --name lan_client --cap-add NET_ADMIN --network lan_net alpine
sudo docker exec lan_client ip route add 172.20.0.0/24 via 172.10.0.2
```

```
manager@srvdmz:~$ sudo docker run -dit --name lan_client --cap-add NET_ADMIN --network lan_net alpine
e0a8b1e83028cb60bf92bbdc49e5ba307655bce1a32c0b7718284e2f2025eb48
manager@srvdmz:~$ sudo docker rm -f lan_client
lan_client
manager@srvdmz:~$ sudo docker run -dit --name lan_client --cap-add NET_ADMIN --network lan_net alpine
bcc405c6a27fac9920d1ef2d4a9d30fdb0a6b5fc67694d5d9a532b3b1176d0e7
manager@srvdmz:~$ sudo docker exec lan_client ip route add 172.20.0.0/24 via 172.10.0.2
```

```
sudo docker run -dit --name wan_client --cap-add NET_ADMIN --network wan_net alpine
```

Vérification : les 4 conteneurs sont actifs (statut Up) :

```
manager@srvdmz:~$ sudo docker rm -f wan_client
wan_client
manager@srvdmz:~$ sudo docker run -dit --name wan_client --cap-add NET_ADMIN --network wan_net alpine
f9ff92fd7ae4bc11fece55609d8eb7eccf5db6927d3a0606b32047d30c5ab0d9
manager@srvdmz:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
f9ff92fd7ae4   alpine    "/bin/sh"               31 seconds ago Up 30 seconds           wan_client
bcc405c6a27f   alpine    "/bin/sh"               About a minute ago Up About a minute       lan_client
52cfad83882c   nginx     "/docker-entrypoint...." 8 minutes ago  Up 8 minutes   80/tcp         web_dmz
8385f078aa9c   debian    "bash"                  2 hours ago    Up 2 hours           firewall
manager@srvdmz:~$
```

Étape 5 : Tests de validation

Test 1 : LAN vers DMZ (HTTP)

Validation de l'architecture en effectuant une requête HTTP depuis le client LAN pour afficher la page du serveur Nginx situé en DMZ.

```
sudo docker exec lan_client apk add curl
sudo docker exec lan_client curl http://172.20.0.3
```

```
manager@srvdmz:~$ sudo docker exec lan_client apk add curl
( 1/10) Installing brotli-libs (1.2.0-r0)
( 2/10) Installing c-ares (1.34.6-r0)
( 3/10) Installing libunistring (1.4.1-r0)
( 4/10) Installing libidn2 (2.3.8-r0)
( 5/10) Installing nghttp2-libs (1.69.0-r0)
( 6/10) Installing nghttp3 (1.13.1-r0)
( 7/10) Installing libpsl (0.21.5-r3)
( 8/10) Installing zstd-libs (1.5.7-r2)
( 9/10) Installing libcurl (8.17.0-r1)
(10/10) Installing curl (8.17.0-r1)
Executing busybox-1.37.0-r30.trigger
OK: 13.2 MiB in 26 packages
manager@srvdmz:~$ sudo docker exec lan_client curl http://172.20.0.3
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0      0     0      0      0      0  --:--:-- --:--:-- --:--:--   0<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
100  896 100  896  0      0 337985      0  --:--:-- --:--:-- --:--:-- 448000
manager@srvdmz:~$
```

Résultat : La page HTML « Welcome to nginx! » s'affiche. Le routage LAN → DMZ via le firewall fonctionne parfaitement.

Test 2 : WAN vers DMZ (HTTP)

Vérification que le WAN peut également accéder au serveur web en DMZ.

```
sudo docker exec wan_client apk add curl
```

```
manager@srvdmz:~$ sudo docker exec wan_client apk add curl
( 1/10) Installing brotli-libs (1.2.0-r0)
( 2/10) Installing c-ares (1.34.6-r0)
( 3/10) Installing libunistring (1.4.1-r0)
( 4/10) Installing libidn2 (2.3.8-r0)
( 5/10) Installing nghttp2-libs (1.69.0-r0)
( 6/10) Installing nghttp3 (1.13.1-r0)
( 7/10) Installing libpsl (0.21.5-r3)
( 8/10) Installing zstd-libs (1.5.7-r2)
( 9/10) Installing libcurl (8.17.0-r1)
(10/10) Installing curl (8.17.0-r1)
Executing busybox-1.37.0-r30.trigger
OK: 13.2 MiB in 26 packages
```

```
sudo docker exec wan_client curl http://172.20.0.3
```

```
manager@srvdmz:~$ sudo docker exec wan_client curl http://172.20.0.3
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Dload  % Upload   Total   Spent    Left  Speed
100  896 100    896    0     0 4<!DOCTYPE html>--:-- --:--:-- --:--:--    0
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
03240    0 --:--:-- --:--:-- --:--:-- 448000
manager@srvdmz:~$
```

Résultat : Le WAN accède aussi au serveur Nginx en DMZ. Le routage WAN → DMZ fonctionne.

Test 3 : Isolation WAN vers LAN

Vérification que le firewall bloque bien le trafic direct du WAN vers le LAN.

```
sudo docker exec wan_client ping -c 3 172.10.0.2
```

```
manager@srvdmz:~$ sudo docker exec wan_client ping -c 3 172.10.0.2
PING 172.10.0.2 (172.10.0.2): 56 data bytes

--- 172.10.0.2 ping statistics ---
3 packets transmitted, 0 packets received, 100% packet loss
manager@srvdmz:~$
```

Résultat : 100% packet loss — le ping échoue. Le firewall bloque correctement le trafic WAN → LAN, prouvant l'efficacité de la règle iptables DROP.

Récapitulatif final

Vue d'ensemble de l'infrastructure déployée : conteneurs, réseaux et règles iptables.

```
manager@srvdmz:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
f9ff92fd7ae4   alpine    "/bin/sh"               23 minutes ago Up 23 minutes          wan_client
bcc405c6a27f   alpine    "/bin/sh"               24 minutes ago Up 24 minutes          lan_client
52cfad83882c   nginx     "/docker-entrypoint. ...." 32 minutes ago Up 32 minutes        80/tcp          web_dmz
8385f078aa9c   debian    "bash"                  3 hours ago   Up 3 hours          firewall

manager@srvdmz:~$ sudo docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
24e581809d91   bridge   bridge    local
eff96c1b520e   dmz_net   bridge    local
2ea651a840da   host     host      local
c0bbaeb6dc3    lan_net   bridge    local
58e7071e692a   none     null      local
6a64aafdfbeb   wan_net   bridge    local

manager@srvdmz:~$ sudo docker exec firewall iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source                   destination

Chain FORWARD (policy ACCEPT 10 packets, 2804 bytes)
 pkts bytes target     prot opt in     out     source                   destination
    14   892 ACCEPT     tcp  --  any    any    anywhere                anywhere            tcp dpt:http
     0     0 DROP      all  --  any    any    172.30.0.0/24            adsl-172-10-0-0.dsl.sndg02.sbcglobal.net/24

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source                   destination

manager@srvdmz:~$
```

4 conteneurs actifs (firewall, web_dmz, lan_client, wan_client), **3 réseaux** (wan_net, dmz_net, lan_net), **2 règles iptables** (ACCEPT HTTP + DROP WAN → LAN). 14 paquets HTTP transités par le firewall.

Analyse Critique et Compétences SISR

Ce projet permet de valider plusieurs compétences clés du référentiel BTS SIO option SISR :

Compétence SISR	Mise en œuvre dans le projet
Gestion du patrimoine informatique	Mise en place d'une architecture réseau segmentée sur VM Proxmox
Sécurisation des accès	Règles iptables, isolation LAN/DMZ/WAN
Répondre aux incidents	Tests de connectivité et validation du routage
Optimisation de l'infrastructure	Architecture conteneurisée sur hyperviseur, portable et reproductible

Apports pédagogiques

- Compréhension concrète de la segmentation réseau (WAN / DMZ / LAN)
- Maîtrise des règles iptables pour le filtrage et le routage de paquets
- Utilisation de Docker comme outil de simulation d'infrastructure réseau
- Déploiement sur hyperviseur Proxmox — environnement professionnel de virtualisation
- Validation par les tests : approche méthodique et reproductible
- Infrastructure as Code : toute l'architecture est reproductible en quelques commandes

Limites et perspectives

- **HTTPS** : En production, le serveur web devrait être accessible uniquement en HTTPS (port 443) avec un certificat TLS.
- **Persistance** : Les règles iptables sont perdues au redémarrage du conteneur ; un script d'initialisation serait nécessaire.
- **Monitoring** : Coupler cette architecture avec une solution de métrologie (Prometheus/Grafana) pour surveiller le trafic.

Annexe A — Plan de tests formalisé

Au-delà des trois tests présentés en section 5, un plan de tests structuré a été exécuté pour valider l'ensemble des règles de filtrage et de routage de l'architecture.

#	Test	Commande	Résultat attendu	Résultat obtenu	Statut
T1	LAN → DMZ (HTTP)	<code>curl http://172.20.0.3</code>	Page Welcome to nginx	Page Welcome to nginx	OK
T2	WAN → DMZ (HTTP)	<code>curl http://172.20.0.3</code>	Page Welcome to nginx	Page Welcome to nginx	OK
T3	WAN → LAN (ICMP)	<code>ping -c 3 172.10.0.2</code>	100% packet loss	100% packet loss	OK
T4	LAN → DMZ (ICMP)	<code>ping -c 3 172.20.0.3</code>	Réponses reçues	Réponses reçues	OK
T5	Persistance des routes	<code>docker exec web_dmz ip route</code>	Routes 172.10/24 et 172.30/24 présentes	Routes présentes	OK
T6	Compteurs iptables	<code>iptables -L -v</code>	Paquets ACCEPT > 0, DROP enregistrés	14 ACCEPT, 3 DROP	OK
T7	Isolation WAN → DMZ port 22	<code>nc -zv 172.20.0.3 22</code>	Refus de connexion (port fermé)	Connection refused	OK

Bilan : 7 tests / 7 réussis. L'architecture répond aux exigences d'isolation et d'accès définies en amont.

Annexe B — Persistance des règles et procédure de rollback

Problématique de persistance

Les règles iptables chargées à chaud (iptables -A) sont perdues au redémarrage du conteneur firewall. En production, il est impératif de garantir leur restauration automatique pour ne pas exposer le LAN.

Solution 1 — Script d'initialisation embarqué

```
fw-init.sh:
iptables -F
iptables -A FORWARD -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -s 172.30.0.0/24 -d 172.10.0.0/24 -j DROP
echo "Firewall rules loaded"
```

Le script est copié dans le conteneur puis exécuté au démarrage via docker exec firewall /fw-init.sh dans un service systemd docker-firewall-init.service.

Solution 2 — iptables-persistent

```
apt install -y iptables-persistent
iptables-save > /etc/iptables/rules.v4
```

Les règles sont sauvegardées dans /etc/iptables/rules.v4 et restaurées automatiquement par le service netfilter-persistent à chaque redémarrage.

Procédure de rollback

En cas d'erreur de configuration ayant coupé tous les flux, la procédure de rollback consiste à reset l'ensemble des chaînes puis redéployer les règles connues bonnes :

```
iptables -P INPUT ACCEPT && iptables -P FORWARD ACCEPT && iptables -P OUTPUT
ACCEPT
iptables -F && iptables -X && iptables -Z
iptables-restore < /etc/iptables/rules.v4.backup
```

Annexe C — Veille et référentiels normatifs

Sources de veille consultées

- Documentation officielle Docker — <https://docs.docker.com>
- Documentation iptables — <https://www.netfilter.org/documentation>
- OWASP — Network Segmentation Cheat Sheet
- ANSSI — Guide de cloisonnement réseau (recommandations RGS)
- Bulletins CERT-FR concernant Docker, Nginx et le noyau Linux

Référentiels et normes mobilisés

Référentiel	Domaine	Application au projet
RFC 1918	Adressage IP privé	Plages 172.10/24, 172.20/24, 172.30/24 issues du bloc 172.16.0.0/12
RFC 9110	HTTP/1.1	Validation par requêtes curl sur le port 80
ISO/IEC 27033-2	Sécurité réseau	Principe de segmentation et de zones de confiance
ANSSI — Guide cloisonnement	Sécurité périmétrique	Architecture WAN/DMZ/LAN avec firewall dédié
OWASP Network Segmentation	Sécurité applicative	Séparation des serveurs publics et internes
RGPD	Données personnelles	Aucune donnée personnelle traitée dans la maquette

Identité et développement professionnel

La maîtrise des concepts de segmentation réseau et de filtrage iptables a été ajoutée au profil LinkedIn et au CV. Le projet est également utilisé comme support de discussion lors d'entretiens techniques (architecture de sécurité périmétrique).

Annexe D — Cartographie détaillée des compétences E5

Mise en correspondance précise des activités du projet avec les sous-compétences et les indicateurs de performance du référentiel BTS SIO 2026.

Compétence E5	Sous-compétences mobilisées	Mise en œuvre dans le projet
Gérer le patrimoine informatique	• Recenser les ressources • Exploiter normes et standards • Vérifier les habilitations • Vérifier la continuité	Plan d'adressage formalisé (document parallèle). Application de RFC 1918, RFC 9110, recommandations ANSSI. Capabilities Docker (NET_ADMIN, NET_RAW) ajustées au strict nécessaire.
Répondre aux incidents	• Diagnostiquer une demande • Traiter les services réseau	Diagnostic systématique par ping, curl, iptables -L -v. Compteurs iptables exploités pour identifier les flux bloqués (Annexe A).
Travailler en mode projet	• Analyser les objectifs • Planifier les activités • Indicateurs de suivi	Découpage en 5 étapes linéaires (création réseaux → firewall → DMZ → clients → tests). Plan de tests formel comme indicateur d'avancement.
Mettre à disposition un service	• Tests d'intégration • Déployer un service • Accompagner	7 tests d'acceptation documentés (Annexe A). Déploiement automatisable. Documentation utilisateur (le présent dossier) et procédure de rollback (Annexe B).
Organiser son développement professionnel	• Veille informationnelle • Identité professionnelle	Veille régulière (Annexe C). Profil LinkedIn enrichi avec les compétences réseau et sécurité.

Conclusion

Ce projet de mise en place d'une architecture DMZ avec Docker sur une machine virtuelle Proxmox démontre qu'il est possible de simuler une infrastructure réseau complète et sécurisée en utilisant des conteneurs sur un hyperviseur professionnel.

La segmentation en trois zones distinctes (WAN, DMZ, LAN) avec un firewall iptables illustre les principes fondamentaux de la sécurité périmétrique. Le déploiement sur Proxmox ajoute une couche de virtualisation représentative des environnements de production.

Les compétences mobilisées — administration réseau, filtrage de paquets, conteneurisation, virtualisation — correspondent directement aux attendus du référentiel SISR et préparent efficacement à l'exercice des fonctions d'administrateur systèmes et réseaux.